508dbf5b-c3ad-4b63-b9a8-7fadf41d77c9

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'styled-components/styled-components' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/styled-components/styled-components (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-01-11T03:35:16.089Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
[WIP] Fix syntax error in styled-components v6 handling
</pr_title>

<pr_description>
- [x] Understand the issue: Invalid CSS syntax (e.g., unbalanced braces in property values) causes stylis to misinterpret closing braces, dropping subsequent styles
- [x] Explore the codebase and understand the CSS processing flow
- [x] Identify the root cause: stylis.compile() interprets extra `}` as closing the rule block
- [x] Design a solution: Add CSS sanitization function that detects and removes malformed declarations while preserving

```
valid ones
- [x] Prototype and test the solution locally
- [x] Implement the fix in `utils/stylis.ts`
- [x] Add tests to verify the fix
- [x] Run existing tests to ensure no regressions (397 web tests
+ 37 native tests pass)
- [x] Create changeset for the fix
- [ ] Run code review and address feedback
- [ ] Run CodeQL security check

<issue_title>styled-components v6: syntax error in one line
prevents remaining styles from applying</issue_title>
&gt;<issue_description>
&gt;
&gt; ## 📝 Description
&gt;
&gt; In **styled-components v6**, if a line inside a styled
block contains **invalid CSS syntax**, it causes all
**subsequent styles to be ignored**.
&gt;
&gt; In **v5**, the same malformed line would be **silently
ignored**, and the rest of the valid styles would still be
applied.
&gt;
&gt; ---
&gt;
&gt; ## 💥 Reproduction Example
&gt;
&gt; ```tsx
&gt; const Circle = styled.div`
>   width: 100px;
>   height: 100px;
>   border-radius: 50%;
>   line-height: ${({ theme }) => "14px}"}; // ⛔ Syntax error:
redundant `}`
>   background-color: green; // ✖ This is ignored in v6
> `;
&gt; ```
&gt;
&gt;
&gt; In v6, because of the syntax error on the line-height line,
background-color: green; is also ignored.
&gt;
&gt; Code example in code sandbox
&gt; https://codesandbox.io/p/sandbox/styled-components-v6-
examples-forked-9hm43z
&gt;
```

> Demo
><img src="https://github.com/user-attachments/assets/7758921c-6620-4d9a-9ea0-b220952949eb">
>
> # Environment
> * styled-components: 5.3.11 -> 6.0.7
> * react: 18.0.0
>
>
> ## Expected Behavior (as in v5)
> - The invalid line is ignored.
> - background-color: green; is still applied.
> - The background should render as a green.
>
>
> ## Actual Behavior in v6
> - The error in line-height causes background-color and any further styles to be ignored entirely.
> - The background does not turn green.
> </issue_description>
>
><agent_instructions>see if there's a pragmatic way of handling this without butchering performance</agent_instructions>
>
> ## Comments on the Issue (you are @copilot in this section)
>
><comments>
></comments>
></pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@quantizor</author>
@copilot explore other classes / variations of this problem such as other types of rule-like incomplete syntax causing broken css. the library's default behavior should be to contain invalid syntax to just the affected line
</comment_old>

<comment_new>
<comment_id>3733939377</comment_id>
<author>@quantizor</author>

```
@copilot resume
</comment_new>
```

</pr_comments>

</comments>

The last **4** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Summary

I've completed the work on fixing CSS syntax error handling in styled-

components v6. Here's what was done:

Issues Fixed
1. **Original Issue**: Unbalanced closing braces (e.g., `line-height: ${() => "14px}"}`) no longer cause all subsequent styles to be ignored
2. **Issue #5613**: JS-style line comments (`//`) in multiline declarations like `calc()` no longer cause parsing errors

Implementation
Added three new functions to `utils/stylis.ts`:
- `stripLineComments()`: Strips `//` comments anywhere in CSS while preserving strings (e.g., `content: "http://..."`)
- `hasUnbalancedBraces()`: Fast O(n) check to detect CSS with brace issues
- `sanitizeCSS()`: Removes only problematic declarations while preserving valid subsequent styles

Performance Considerations
- **Fast path**: Valid CSS passes through unchanged with minimal overhead (single O(n) scan)
- **Slow path**: Only triggered when issues are detected, removes only affected declarations
- Addressed code review feedback for performance (caching `css.length`, consistent bounds checking)

Tests
Added 16 comprehensive inline snapshot tests covering:
- Unbalanced closing braces in various positions
- Multiple malformed declarations
- Braces inside quoted strings (preserved correctly)
- Line comments at start of line, end of line, and inside multiline declarations
- URLs in strings preserved (e.g., `http://`)

Results
- All 404 web tests pass
- All 37 native tests pass
- CodeQL security check: no issues found

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'styled-components/styled-

components' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/styled-components/styled-components (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-01-11T03:49:03.275Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
[WIP] Fix syntax error in styled-components v6 handling
</pr_title>

<pr_description>
- [x] Understand the issue: Invalid CSS syntax causes stylis to misinterpret closing braces, dropping subsequent styles
- [x] Explore the codebase and understand the CSS processing flow
- [x] Identify the root cause: stylis.compile() interprets extra `}` as closing the rule block
- [x] Design a solution: Add CSS sanitization that contains invalid syntax to just the affected declaration
- [x] Implement the fix in `utils/stylis.ts`:
  - `stripLineComments()`: Strips JS-style `//` comments anywhere in CSS (fixes #5613)
  - `hasUnbalancedBraces()`: Fast check for CSS with brace issues
  - `sanitizeCSS()`: Removes only problematic declarations while preserving valid ones
- [x] Add comprehensive inline snapshot tests for all variations
- [x] Run existing tests to ensure no regressions (404 web tests + 37 native tests pass)
- [x] Update changeset
- [x] Run code review and address feedback
- [x] Run CodeQL security check - no issues found

<issue_title>styled-components v6: syntax error in one line prevents remaining styles from applying</issue_title>
><issue_description>

```
&gt;
&gt; ## 📝 Description
&gt;
&gt; In **styled-components v6**, if a line inside a styled
block contains **invalid CSS syntax**, it causes all
**subsequent styles to be ignored**.
&gt;
&gt; In **v5**, the same malformed line would be **silently
ignored**, and the rest of the valid styles would still be
applied.
&gt;
&gt; ---
&gt;
&gt; ## 💥 Reproduction Example
&gt;
&gt; ```tsx
&gt; const Circle = styled.div`
>   width: 100px;
>   height: 100px;
>   border-radius: 50%;
>   line-height: ${({ theme }) => "14px}"}; // ⛔ Syntax error:
redundant `}`
>   background-color: green; // ✕ This is ignored in v6
> `;
&gt; ```
&gt;
&gt;
&gt; In v6, because of the syntax error on the line-height line,
background-color: green; is also ignored.
&gt;
&gt; Code example in code sandbox
&gt; https://codesandbox.io/p/sandbox/styled-components-v6-
examples-forked-9hm43z
&gt;
&gt; Demo
&gt;<img src="https://github.com/user-attachments/assets/
7758921c-6620-4d9a-9ea0-b220952949eb">
&gt;
&gt; # Environment
&gt;  * styled-components: 5.3.11 -&gt; 6.0.7
&gt;  * react: 18.0.0
&gt;
&gt;
&gt; ## Expected Behavior (as in v5)
&gt; - The invalid line is ignored.
&gt; - background-color: green; is still applied.
&gt; - The background should render as a green.
```

```
&gt;
&gt;
&gt; ##  Actual Behavior in v6
&gt; - The error in line-height causes background-color and any
further styles to be ignored entirely.
&gt; - The background does not turn green.
&gt; </issue_description>
&gt;
&gt;<agent_instructions>see if there's a pragmatic way of
handling this without butchering performance</
agent_instructions>
&gt;
&gt; ## Comments on the Issue (you are @copilot in this section)
&gt;
&gt;<comments>
&gt;</comments>
&gt;</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@quantizor</author>
@copilot explore other classes / variations of this problem such
as other types of rule-like incomplete syntax causing broken
css. the library's default behavior should be to contain invalid
syntax to just the affected line
</comment_old>

<comment_old>
<author>@quantizor</author>
@copilot resume
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot resume

Resumed and completed the work. In commit `5ae49f0` I:

1. Explored additional CSS syntax variations (extra opening
braces, unclosed parentheses, missing colons, extra semicolons,
etc.)
```

2. Fixed issue #5613 - line comments (`//`) in multiline declarations like `calc()` now properly stripped
3. Added 16 comprehensive inline snapshot tests covering all variations
4. Ensured invalid syntax is contained to just the affected line - subsequent valid styles are preserved

All 404 web tests and 37 native tests pass. CodeQL security check found no issues.
</comment_old>

<comment_new>
<comment_id>3733963385</comment_id>
<author>@quantizor</author>
@copilot try to further optimize sanitizeCSS as it is the hottest path in the system. analyze branching complexity, short circuiting optimizations, polymorphism etc. Use microbenchmarks to validate approaches.
</comment_new>

</pr_comments>

</comments>

The last **6** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.